Pruning and Distilling a Mixture-of-Experts Language Model for TypeScript Specialization

Sartaj Bhuvaji [1]

Abstract

We apply the Minitron structured pruning and knowledge distillation recipe (Muralidharan et al., 2024; Sreenivas et al., 2024) to a Mixture-of-Experts (MoE) language model, Qwen3-30B-A3B-Base, and specialize the result toward TypeScript. Both Minitron papers prune only dense transformers; we extend their forward-pass-only, activation-based importance-estimation philosophy to a pruning axis neither paper covers – expert pruning. We find that a gate-weighted expert-output-norm criterion (independently converged on by REAP (Lasby et al., 2025) and NVIDIA's own Minitron-MoE extension) correctly identifies previously-reported "Super Experts" (Anonymous, 2025) as the top-ranked expert in their respective layers, and that a cheap, single-forward-pass depth-pruning proxy (Block Importance) correlates strongly (Pearson $r = 0.91$) with the expensive, 49-forward-pass perplexity-based criterion from the original Minitron paper, agreeing exactly on the three most critical layers. Acting on this scoring, not just ranking by it, we run the first real pruning experiment in this project: a sweep over nine layer counts and four selection strategies (BI-greedy, perplexity-greedy, best-contiguous-block, and a random control) shows principled selection beating random by close to an order of magnitude once more than a handful of layers are pruned, and finds contiguous-block removal – the second Minitron paper's own preferred depth-pruning approach – to be a genuinely mixed result on this model rather than a clean win. This is a living document, updated as further pruning, distillation, and retraining experiments complete; no head or expert has been pruned yet, and no retraining has occurred.

[1]Seattle University, Seattle, WA, USA. Correspondence to: Sartaj Bhuvaji <sbhuvaji@seattleu.edu>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

1. Introduction

Minitron (Muralidharan et al., 2024) demonstrated that a large language model can be compressed into a family of smaller models via structured pruning followed by knowledge distillation, using up to 40× fewer training tokens than training each size from scratch while matching or exceeding the accuracy of from-scratch baselines. A follow-up paper (Sreenivas et al., 2024) applied the same recipe under a more realistic constraint – no access to the original pretraining data – and introduced teacher correction and a downstream-task-based depth-pruning criterion.

Both papers prune exclusively dense transformers. Every weight in a dense model's MLP is applied to every token that passes through it. This is not true of a Mixture-of-Experts (MoE) model: an MoE layer routes each token to a small subset of experts (commonly 8 of 128, in the model we study here), so most of the model's parameters are never touched by any given token. This work asks what happens when the Minitron recipe is applied to a model with this property, and in particular, how to score the importance of an axis – expert pruning – that has no precedent in either source paper.

Our target model is Qwen3-30B-A3B-Base (Qwen Team, 2025): 48 decoder layers, 32 query / 4 key-value attention heads (GQA), 128 routed experts per layer with 8 active per token, no shared expert, approximately 30.5B total parameters of which approximately 3.4B are active for any given token. We specialize this model toward TypeScript, using bigcode/the-stack-dedup's TypeScript split (Kocetkov et al., 2022) for both calibration and retraining data, drawn from disjoint parquet shards so the two pools never overlap.

This paper is written incrementally, alongside the

project's own commit history: each section is updated with real, measured results only, once the corresponding experiment has actually run. As of this writing, depth pruning has been characterized (Section 6) but not yet combined with the other axes, retrained, or finalized to one candidate; attention-head and expert pruning have been scored but not yet acted on.

2. Related Work

Minitron. Muralidharan et al. (2024) prune Nemotron-4 15B to 8B and 4B using purely activation-based importance estimation (no gradients), scoring attention heads, MLP neurons, and embedding channels via a shared formula that aggregates activations over a small (1024-sample) calibration set, and scoring depth separately via perplexity delta or Block Importance (cosine distance between a layer's input and output). Sreenivas et al. (2024) extend this to settings without the original pretraining data, introducing teacher correction (lightly fine-tuning the teacher on the actual distillation data before pruning) and a downstream-task-based depth-pruning criterion using contiguous-block removal, evaluated on Llama 3.1 8B and Mistral NeMo 12B.

Expert pruning in MoE models. Neither Minitron paper addresses Mixture-of-Experts models. REAP (Lasby et al., 2025) scores expert importance as a gate-weighted expert-output norm, averaged over each expert's routed tokens, and evaluates directly on Qwen3-30B-A3B (the model we study). NVIDIA's Model-Optimizer library independently implements the same formula as part of its own Minitron-MoE extension, used to compress Nemotron-3-Nano-30B-A3B. Prior work on Mixtral-scale models found selection-frequency-based criteria underperform output-magnitude-based ones (Lu et al., 2024), consistent with the choice we adopt (Section 5.2).

Super Experts. Anonymous (2025) report that Qwen3-30B-A3B contains a small number of experts (three in the first four layers of the model, by their numbering) whose rare but extremely large-magnitude outputs anchor attention sinks; ablating them alone raises WikiText-2 perplexity from 8.70 to 59.86. We treat this as a regression test for our own expert importance metric (Section 5.2).

3. Target Model and Data

3.1. Model

Qwen3-30B-A3B-Base (Qwen Team, 2025) is a base (not instruction-tuned) causal language model with 48 Qwen3MoeDecoderLayer blocks, hidden size 2048, 32 query heads and 4 key-value heads (grouped-query attention, head_dim 128), and 128 routed experts per layer (Qwen3MoeExperts, moe_intermediate_size 768) with the top 8 active per token, no shared expert. Every layer is sparse (decoder_sparse_step=1, mlp_only_layers empty). Experts within a layer are not separate submodules but two batched parameter tensors (gate_up_proj, down_proj), with the expert index as the leading dimension – pruning an expert means slicing that dimension.

Table 1. Baseline evaluation of Qwen3-30B-A3B-Base, unpruned.

Task	Setting	Score
HumanEval-TS	pass@1	0.6708
MBPP-TS	pass@1	0.7333
MMLU	5-shot, acc	0.8101
ARC-Challenge	25-shot, acc	0.6536
ARC-Challenge	25-shot, acc_norm	0.6749
Winogrande	5-shot, acc	0.7798

3.2. Data

Calibration and retraining data are both drawn from bigcode/the-stack-dedup's data/typescript split (71 parquet shards). The first shard is reserved for calibration, the remaining 70 for retraining; because the split is shard-level rather than row-level, the two pools are disjoint by construction, which we additionally verify empirically against each sample's hexsha.

4. Baseline Evaluation of the Unpruned Teacher

Before any pruning decision, we establish the unpruned model's performance on the evaluation suite every subsequent pruning candidate will be measured against. Following both Minitron papers' evaluation settings (temperature 0.2, nucleus sampling top-p = 0.95 for generation tasks), we report:

HumanEval-TS and MBPP-TS (both MultiPL-E TypeScript translations) are treated as the primary, gating tier, since they measure the capability this project specializes toward. MMLU, ARC-Challenge, and Winogrande are tracked but not optimized for, giving a comparison point against both Minitron papers' own general-knowledge tables without over-indexing on general capability at the expense of TypeScript performance.

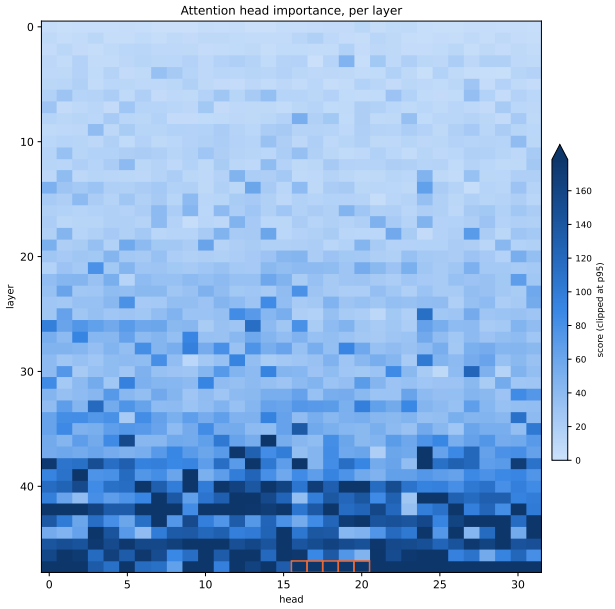


Figure 1. Head importance (F_h, Equation (1)), all 48 layers × 32 heads. Color is clipped at the 95th percentile (outlined cells mark the true top 5) since a linear scale to the true maximum leaves the rest of the grid indistinguishable from zero.

5. Importance Estimation

Both Minitron papers score importance from forward-pass activations only, never gradients, over a small calibration set. We follow the same philosophy for every axis, including the one neither paper covers.

5.1. Attention Heads

Head importance uses the original formula unchanged. For head h in a given layer, let $a_{t,h} \in \mathbb{R}^{d_{\text{head}}}$ be that head's output for calibration token t, prior to the W^O merge. The per-token score is its L_2 norm, aggregated over every token t in the calibration set:

$$F_h = \text{agg}_t \left(\|a_{t,h}\|_2 \right), \tag{1}$$

where agg is a pluggable aggregation function – Muralidharan et al. (2024) ablate mean-of-absolute-value, L_2 norm, and variance, finding the best choice is not the same for every axis; we use the L_2 variant throughout this paper. Across all 48 layers (48 calibration samples, mean length 177 tokens), scores range from 2.39 to 967.72, with the single largest value at the model's final layer (layer 47) – visible as the dark band at the bottom of Figure 1.

Algorithm 1 Expert importance from routing data

Input: expert indices $e_1, \ldots, e_N$, gate weights $g_1, \ldots, g_N$, expert outputs $f_1, \ldots, f_N \in \mathbb{R}^d$ for every (token, selected-expert) pair in the calibration set; number of experts E
sum$[1..E] \leftarrow 0$; count$[1..E] \leftarrow 0$
for $i = 1$ to N do
  sum$[e_i] \leftarrow$ sum$[e_i] + g_i \cdot \|f_i\|_2$
  count$[e_i] \leftarrow$ count$[e_i] + 1$
end for
for $j = 1$ to E do
  $S_j \leftarrow$ count$[j] > 0$? sum$[j]$/count$[j]$: 0
end for
Return: $S_1, \ldots, S_E$

5.2. Experts

Neither Minitron paper offers a formula for this axis. We adopt $S_j = \text{mean}_{x \in X_j} [g_j(x) \cdot \|f_j(x)\|_2]$, where X_j is the set of tokens routed to expert j, $g_j(x)$ is that token's renormalized routing weight, and $f_j(x)$ is the expert's output – the formula independently reached by both REAP (Lasby et al., 2025) and NVIDIA Model-Optimizer's Minitron-MoE extension, in preference to two alternatives considered (selection frequency alone; router logit magnitude alone), both reported to underperform output-magnitude-based scoring in prior work (Lu et al., 2024).

Unlike head importance, there is no dense batch/sequence grid to aggregate over: routing is sparse and data-dependent per token, so S_j is computed from a flat list of (token, expert) assignments rather than a fixed-shape tensor, mirroring how the model's own expert-dispatch loop iterates (Algorithm 1).

An expert with zero routed tokens in the calibration set scores 0 and ranks lowest by construction – appropriate for a calibration-set-specific score, but a real risk if a rare-but-critical expert happens to be undersampled by chance, which is exactly what the Super Experts check below guards against.

As a validation check, not a designed experiment: all three Super Experts reported by Anonymous (2025) for this exact model (layer 1 / expert 68, layer 2 / expert 92, layer 3 / expert 82) rank first among all 128 experts in their respective layers under this metric, on real calibration data, at two different sample sizes (24 and 48 samples) – the three isolated bright cells in Figure 2. On average across all 48 layers, the top ∼16% of experts (20 of 128) already account for roughly a third of a layer's total importance mass (Figure 3), indicating substantial redundancy among the remaining

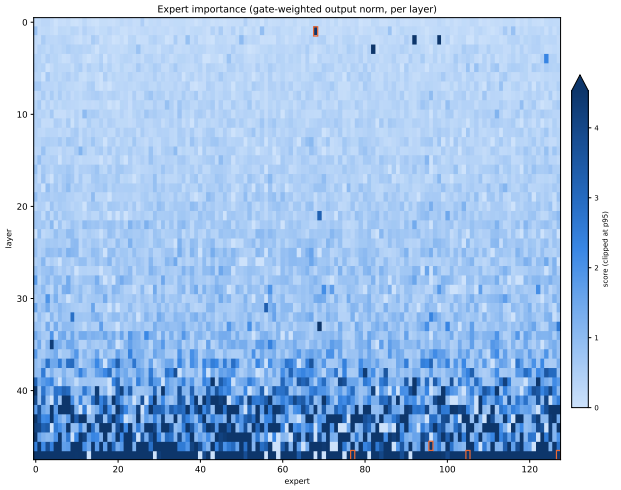


Figure 2. Expert importance (S_j), all 48 layers × 128 experts, same clipping convention as Figure 1. The three outlined cells in layers 1–3 are the Super Experts (Anonymous, 2025).

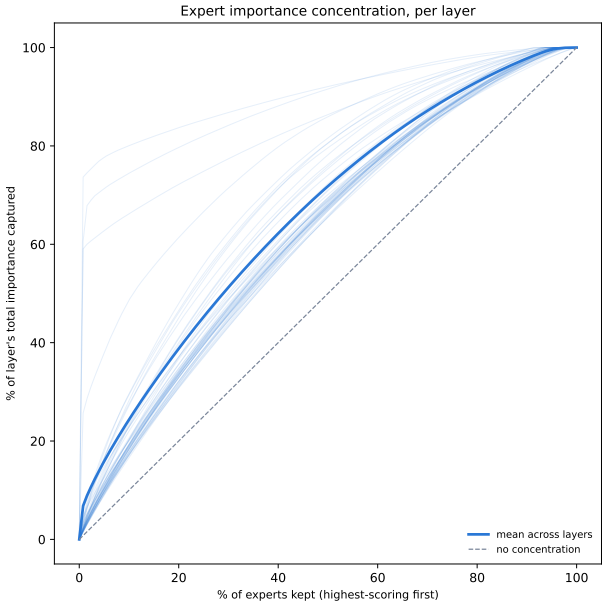


Figure 3. Per-layer cumulative importance captured by the top-k highest-scoring experts (thin lines: all 48 layers; bold: mean). The dashed diagonal is the no-concentration reference.

experts as a candidate for pruning.

5.3. Depth: Block Importance and Perplexity

We implement both depth-scoring criteria from Muralidharan et al. (2024). Block Importance (BI) for layer ℓ, with input activations X_ℓ and output activations $X_{\ell+1}$ over calibration tokens t, is

$$\mathrm{BI}_\ell = 1 - \mathbb{E}_t\left[\frac{X_{\ell,t}^\top X_{\ell+1,t}}{\|X_{\ell,t}\|_2 \|X_{\ell+1,t}\|_2}\right], \tag{2}$$

computable in the same forward pass that produces the two activations. The perplexity-delta criterion instead removes one layer at a time and measures the resulting change in held-out loss relative to the unmodified baseline:

$$\Delta_\ell = \mathcal{L}(\theta \setminus \ell) - \mathcal{L}(\theta), \tag{3}$$

where $\mathcal{L}$ is mean cross-entropy loss over the calibration set and $\theta\setminus\ell$ denotes the model with layer ℓ temporarily skipped (Algorithm 2). This costs $L+1$ forward passes for an L-layer model, versus BI's one, and is run on a smaller calibration sample (16 samples, vs. 48 for BI) for that reason.

Both criteria independently identify the same pattern: the first and last layers of the model matter far more than the middle two-thirds (Figure 4). BI's highest score is at layer 0 (0.488) and second-highest at layer 47 (0.415); its lowest is layer 12 (0.011). The perplexity criterion shows the same edges-matter-most shape at a far more extreme scale – layer 0's removal costs 1.99 nats of cross-entropy, next-highest is layer 47 at 0.60 nats, and the lowest (layer 14) is statistically indistinguishable from zero (−0.0003 nats).

Algorithm 2 Perplexity-delta depth scoring

Input: model θ with layers $1, \ldots, L$; calibration batches $\mathcal{D}$
base $\leftarrow$ MeanLoss$(\theta, \mathcal{D})$
for $\ell = 1$ to L do
  replace layer ℓ's forward with the identity function
  $\Delta_\ell \leftarrow$ MeanLoss$(\theta, \mathcal{D})$ − base
  restore layer ℓ's original forward function
end for
Return: $\Delta_1, \ldots, \Delta_L$

Across all 48 layers, the Pearson correlation between BI and the perplexity delta is $r = 0.91$ (Figure 5), and the two criteria pick the identical top-3 most critical layers (0, 47, 1). Given the 49× cost difference between the two, this agreement is the paper's most directly actionable early finding: Block Importance can be trusted as a cheap stand-in for perplexity-based scoring when iterating over many candidates later in this project, reserving the expensive criterion for confirming a final choice rather than for every candidate evaluated along the way.

Figure 6 places the three cheap, single-forward-pass methods – heads, experts, BI – on the same (min-max normalized) axis; Figure 5 above already gives

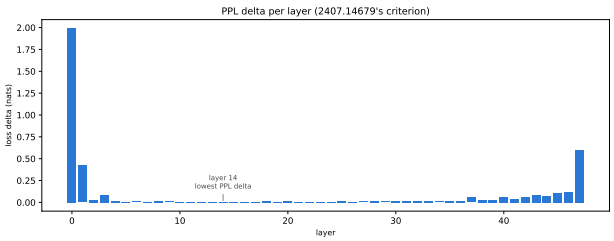


Figure 4. Perplexity delta Δ_ℓ (Equation (3)) per layer.

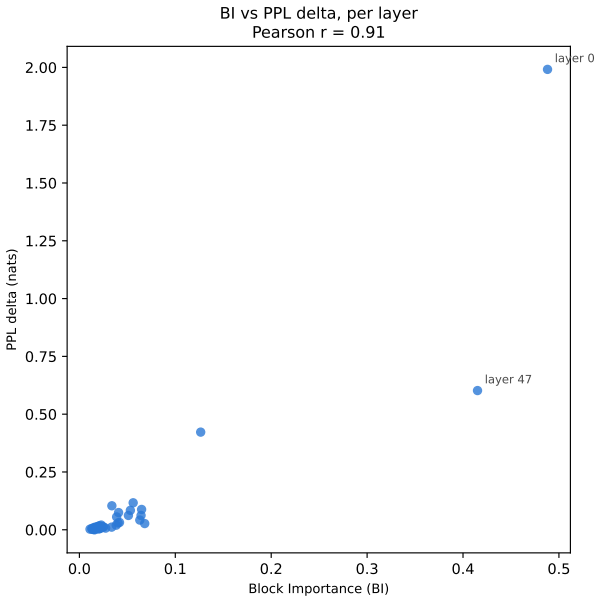


Figure 5. BI (Equation (2)) vs. perplexity delta (Equation (3)), one point per layer.

the fourth, perplexity, directly against BI. Heads and experts agree closely with each other throughout the model; BI diverges from both through the middle third, agreeing with them only at the two edges – the same edges where Figure 5 shows it also agrees with perplexity.

6. Depth Pruning: Zero-Shot Loss vs. Layers Pruned

Sections 5.2 and 5.3 only score which components matter; this section actually prunes them. bonsai.pruning.depth.prune_layers permanently removes a chosen set of decoder layers – unlike skip_layer's temporary identity-swap used for scoring – and bonsai.importance.depth.mean_loss measures the resulting model's zero-shot loss, here on 32 fresh calibration samples (seed 1, a disjoint draw from every prior calibration slice in this paper).

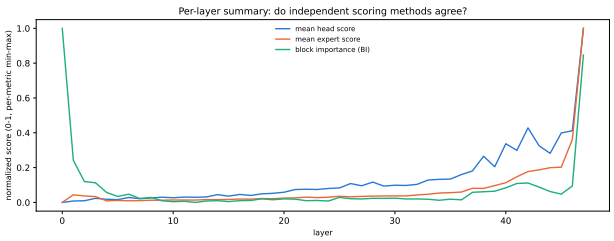


Figure 6. Mean head score, mean expert score, and BI, per layer, min-max normalized within each method.

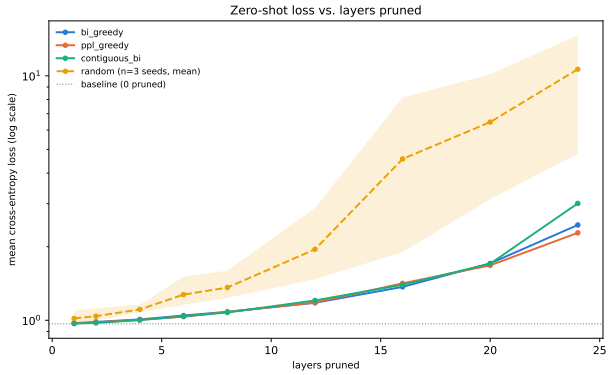


Figure 7. Zero-shot loss vs. layers pruned. Random shown as its 3-seed mean (dashed) and min-max band. Log-scale y-axis.

We sweep nine layer counts ($n \in \{1, 2, 4, 6, 8, 12, 16, 20, 24\}$ of 48) under four selection strategies: bi_greedy (the n lowest-BI layers, Equation (2)); ppl_greedy (the n lowest-perplexity-delta layers, Equation (3)); contiguous_bi (the single contiguous block of n layers with the lowest summed BI – testing Sreenivas et al. (2024)'s own claim, citing Gromov et al., that contiguous removal beats scattered removal); and random (n random layers, 3 seeds – the control every principled strategy above has to actually beat, not merely correlate well with). Each candidate costs exactly one forward pass, since the ranking is already decided; this is what makes a nine-count, four-strategy sweep affordable.

Principled selection clearly beats random at every count tested, by close to an order of magnitude once more than a handful of layers are pruned: at $n = 24$, bi_greedy/ppl_greedy loss is 2.45/2.28 against a random range of 4.81–14.65 across the three seeds (Table 2). bi_greedy and ppl_greedy track each other closely at every count – not merely in the ranking correlation already established ($r = 0.91$, Section 5.3) but in the actual pruning outcome – reinforcing BI as the working criterion for iterating on candidates.

Table 2. Loss increase over baseline (0.9670), by strategy and layer count.

n	bi_gr.	ppl_gr.	contig.	rand. (mean)
1	+0.3%	+0.9%	+0.3%	+5.3%
2	+1.8%	+1.4%	+0.9%	+7.5%
4	+4.3%	+3.8%	+3.6%	+14.5%
6	+8.1%	+7.0%	+7.3%	+31.7%
8	+12.0%	+11.9%	+11.3%	+40.7%
12	+21.9%	+22.2%	+24.6%	+101.8%
16	+41.6%	+46.4%	+44.5%	+372.6%
20	+76.7%	+73.5%	+76.2%	+568.8%
24	+153.8%	+135.6%	+211.0%	+1000.1%

Contiguous-block removal is a genuinely mixed result on this model, not a clean win either way. It is competitive – marginally best – at small n ($n = 2$: loss 0.9755 vs. bi_greedy's 0.9845), but falls behind both greedy strategies as n grows, most visibly by $n = 24$ (211.0% vs. 135.6–153.8%). A plausible reading: BI's lowest-scoring layers already cluster in a contiguous run in the middle of the model (Figure 6), so for small n the two approaches pick nearly the same layers anyway; forcing strict contiguity only starts costing something once n exceeds what that natural cluster contains, pushing the contiguous search into a wider, less-optimal window. We report this as an honest, mixed result rather than adopting whichever framing looks cleaner, consistent with this paper's general practice of reporting ablations as measured (Section 7).

7. Discussion and Open Questions

Several design decisions remain open and are logged here rather than resolved prematurely:

- A specific target layer count. Section 6 characterizes the cost curve but does not commit to one candidate to carry into later axis-combination and retraining – that choice depends on a compute/accuracy tradeoff not yet made.
- Downstream-depth-pruning proxy. Sreenivas et al. (2024)'s preferred depth criterion ranks contiguous layer blocks by downstream-task accuracy after removal (Winogrande, in their case), not by BI or perplexity. No TypeScript equivalent of Winogrande exists; HumanEval-TS or MBPP-TS pass-rate on partially-pruned models is the direct analogue but expensive per candidate block, and a cheaper token-level-accuracy proxy is under consideration.
- Uniform vs. per-layer expert counts. We currently plan uniform expert counts per layer for the first pruning pass, following published evidence that non-uniform allocation did not outperform uniform allocation on this specific model in at least one independent evaluation, though this has not yet been tested within this project directly.
- Interaction between axes. Muralidharan et al. (2024) find that depth and width importance rankings can flip after retraining relative to their zero-shot rankings; whether the same holds when experts are combined with depth and width has no precedent to draw on and will be tested empirically once single-axis pruning is complete.

No results are reported for these open questions; they are listed to make the project's actual state legible, not to imply an answer.

Acknowledgements

This is an independent research project with no external funding.

References

Anonymous. Superweight and super experts in large language models. arXiv preprint arXiv:2507.23279, 2025.

Kocetkov, D., Li, R., Allal, L. B., et al. The stack: 3 tb of permissively licensed source code. https://huggingface.co/datasets/bigcode/the-stack-dedup, 2022.

Lasby, M. et al. Reap: Router-weighted expert activation pruning for mixture-of-experts language models. arXiv preprint arXiv:2510.13999, 2025.

Lu, X. et al. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. arXiv preprint arXiv:2402.14800, 2024.

Muralidharan, S., Sreenivas, S. T., Joshi, R., Chochowski, M., Patwary, M., Shoeybi, M., Catanzaro, B., Kautz, J., and Molchanov, P. Compact language models via pruning and knowledge distillation. arXiv preprint arXiv:2407.14679, 2024.

Qwen Team. Qwen3 technical report. https://huggingface.co/Qwen/Qwen3-30B-A3B-Base, 2025.

Sreenivas, S. T., Muralidharan, S., Joshi, R., Chochowski, M., Patwary, M., Shoeybi, M., Catanzaro, B., Kautz, J., and Molchanov, P. Llm pruning and distillation in practice: The minitron approach. arXiv preprint arXiv:2408.11796, 2024.